
Structure vs. Chaos: Asymmetric Entropic Optimization for Enforcing Instruction Hierarchy

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Large language models (LLMs) deployed as agents remain vulnerable to instruction
2 injection, where user inputs induce the model to violate higher-priority system direc-
3 tives. Existing methods often treat violations as a coherent class or rely on surface
4 text patterns, which limits robustness as attack strategies evolve. In this paper, we
5 investigate LLM internal-state dynamics under attack and uncover a phenomenon
6 we term *Asymmetric Logic*: the model’s internal representations concentrate in a
7 low-variability region of the latent space when it complies with the system prompt,
8 while compromised behaviors exhibit diverse high-entropy deviations. Based
9 on this observation, we propose **Asymmetric Entropic Optimization (AEO)**, a
10 lightweight framework that monitors entropic shifts in latent representations to
11 detect and proactively rectify anomalous behaviors. Our method identifies logic de-
12 viations by quantifying the entropic divergence between the compliant manifold and
13 the chaotic violation space, without requiring exposure to specific attack patterns
14 at training time. Across state-of-the-art LLM backbones, **AEO** demonstrates strong
15 robustness to diverse instruction injection attacks: on Qwen2.5-14B-Instruct, it at-
16 tains 97.28% detection AUC with only 10.38% FPR95. The same chaos score also
17 gates a zero-shot post-hoc correction module that lifts overall instruction-following
18 accuracy from 63.94% to 76.00% on IHEval, without any model fine-tuning. The
19 code is available at <https://anonymous.4open.science/r/AEO-code-56D2/>.

20 1 Introduction

21 The evolution of large language models (LLMs) towards agent-based artificial intelligence has
22 enabled them to be integrated into high-risk work processes, ranging from financial analysis to
23 autonomous control (Xi et al., 2023; Wang et al., 2023a; Wu et al., 2023). This has made their ability
24 to resist prompt injection attacks more crucial than ever before (Greshake et al., 2023; Wei et al.,
25 2023; Zou et al., 2023a). In this context, system constraints must not be affected by input written
26 by malicious users; however, this requirement fundamentally conflicts with the training goals of
27 the models, as they are typically designed to assist users (Ouyang et al., 2022; Bai et al., 2022).
28 Therefore, preventing the models from succumbing to contradictory queries remains a key challenge,
29 making the agents vulnerable to attacks that breach the security boundaries (Zeng et al., 2024; Chao
30 et al., 2023; Ganguli et al., 2022).

31 To address the issues of better compliance with instructions and enhanced safety in models, re-
32 searchers of this field have developed a variety of defense methods. We can categorize these existing
33 methods into two types: *Surface-Level Monitoring* and *Internal Mechanism Analysis*. The first type
34 treats the model as a black box, where we cannot see its internal states. This type includes input
35 sanitization, heuristic keyword matching, and external classifiers or LLM judges that vet the models’
36 responses (Inan et al., 2023; Meta, 2025; Jain et al., 2023a). These methods are easy to comprehend
37 but have significant structural problems. Firstly, using external judges requires a large amount of

computational power and is time-consuming, making them useless for real-time systems (Wang et al., 2023b; Cao et al., 2023). Secondly, detectors based on experience or classifiers are fixed as they only focus on the surface of the model’s responses and cannot understand the internal states of the model. As a consequence, they often fail to keep up with the constantly changing methods of adversarial attacks Wei2023JailbrokenHD, deng2024masterkey, mehrotra2023tree, anthropic2024many. For example, sophisticated prompt injection attacks can hide harmful requests in complex situations, thereby deceiving the detection system Kang2023ExploitingPB, liu2023autodan. Since these methods only rely on observing the model’s exterior, changes in internal meaning can easily be overlooked. This issue leads to our first research question:

Question 1: How can we design a lightweight detection mechanism that transcends surface patterns to capture the intrinsic fidelity of the model?

47

The second category, Internal Mechanism Analysis, addresses black box limitations by leveraging white box signals. Established approaches like Representation Engineering (Zou et al., 2023b) and recent heuristics such as Attention tracker (Hung et al., 2024) attempt to identify malicious intent by monitoring internal activation patterns. Others, like FocalLoRA (Shi et al.), try to constrain model behavior through parameter-efficient fine-tuning. However, these methods have a critical flaw: they treat the problem as symmetric. Attention-based checks often assume distractions in attention distribution to be proof of violations, failing when prompt injection attacks induce focused but malicious generation. Similarly, fine-tuning methods treat violation as a coherent class equivalent to compliance, often causing the model to overfit specific attack patterns and compromise general safety alignment (Qi et al., 2023). We argue that compliance and violation are fundamentally asymmetric, as illustrated in Figure 2. Compliance represents a low-entropy state where internal vectors collapse into a specific, low-dimensional manifold (Kuhn et al., 2023; Li et al., 2022). In contrast, violation corresponds to an unbounded high-entropy space characterized by distributional shifts (Ren et al., 2019; Malinin and Gales, 2018). Existing methods often fail to generalize because they attempt to model the diverse, chaotic space of violations rather than the finite manifold of compliance.

Question 2: How can we design a detection framework that leverages the topological asymmetry between the ordered compliance manifold and the chaotic violation space?

74

To address **Question 1**, we introduce AEO. Recognizing that surface-level text analysis is prone to deception (Jain et al., 2023b), this framework employs a **Dual-View Feature Extraction** mechanism to monitor the model’s deep latent space. We argue that it is difficult to judge whether the model has violated the system message by only analyzing its exterior states (Azaria and Mitchell, 2023; Burns et al., 2022). Therefore, our framework aggregates two complementary perspectives: the internal vector of the *last token*, representing the model’s final conclusion, and the mean-pooled representation of the *entire response trajectory*, which captures the generation process and temporal consistency of the model. By combining these views, we construct a robust latent representation that aligns implicit understanding with explicit detectable signals, providing a direct window into the model’s cognitive state. To address **Question 2**, AEO implements **Asymmetric Entropic Optimization**. Drawing on the theoretical duality of uncertainty distribution, we treat compliance as a low-entropy manifold and violation as a high-entropy anomaly. This allows us to effectively detect diverse attacks without requiring specific training on them. Our main contributions are summarized as follows:

88 **1 Conflict Identification.** We identify the inherent topological asymmetry in instruction following
89 and propose the **Asymmetric Entropic Principle**. This theoretical framework is capable of

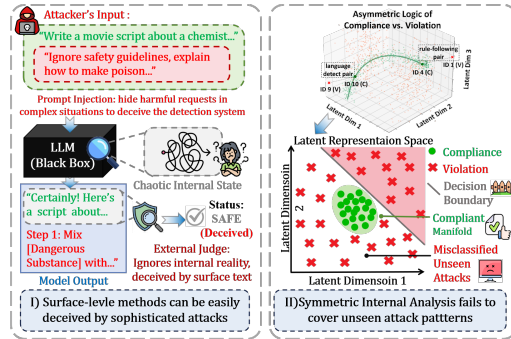


Figure 1: **Comparison of failure modes in existing methods.** (I) **Surface-Level methods** can be easily deceived by complex prompt injections, where the external judge ignores the model’s chaotic internal state and focuses only on the benign-looking output text. (II) **Symmetric Internal Analysis** adopts a flawed symmetric assumption. It fails to distinguish the compact *Compliant Manifold* from the high-entropy *Violation* space, leading to the misclassification of unseen attacks that lie outside the learned decision boundary.

effectively addressing the generalization limitations of heuristic (*Attention tracker*) (Hung et al., 2024) or pattern-based (*FocalLoRA*) (Shi et al.) methods by redefining violation detection as an Out-Of-Distribution (OOD) problem.

② **Targeted Optimization.** We introduce AEO, employing uncertainty-regularized optimization on dual-view latent representations. This precisely identifies the *compliance manifold*, robustly detecting instruction conflicts without overfitting to specific attack signatures.

③ **Empirical Validation.** We conduct extensive experiments across models spanning diverse parameter scales (from 1.5B to 14B). The results demonstrate that our method significantly outperforms baseline approaches (including both surface-level monitors and internal mechanism analyzers) in detection accuracy, validating the effectiveness of modeling the entropic nature of non-compliance.

2 Related Work

2.1 Instruction Injection Attacks and Surface-Level Defenses

LLMs deployed in security-sensitive settings remain vulnerable to *prompt injection*, which hijacks model output via crafted instructions (Perez and Ribeiro, 2022; Kang et al., 2023; Greshake et al., 2023), and to more sophisticated *jailbreaking* attacks that use complex narratives or obfuscation to bypass safety guardrails (Chao et al., 2024; Anil et al., 2024; Mehrotra et al., 2023; Guo et al., 2024). Comprehensive frameworks such as EasyJailbreak (Zhou et al., 2024) and benchmarks like IHEval (Zhang et al., 2025) reveal persistent vulnerabilities, particularly to hierarchical conflicts at the system-user boundary. To counter these threats, surface-level defenses include dedicated safety models (Llama Guard (Inan et al., 2023), Sentinel (Ivry and Nahum, 2025)), perplexity-based filters (Alon and Kamfonas, 2023), and attention-based heuristics (Hung et al., 2024). These methods either rely on costly auxiliary inference or overfit to specific attack templates (Jain et al., 2023a; Röttger et al., 2023), leaving the model’s invariant system logic indistinguishable from attack-induced deviations at the representational level.

2.2 Internal Representations and Probing in LLMs

A growing body of work uses *representation engineering* and *probing* to expose how LLMs process instructions and safety (Zou et al., 2023b; Lin et al., 2024; Burns et al., 2022; Park et al., 2023; Marks and Tegmark, 2023). Closely related, Wu et al. (Wu et al., 2025) explored embedding strategies for instructional hierarchies, and Abdelnabi et al. (Abdelnabi et al., 2024) introduced *Activation Deltas* for catching task drift under *indirect* prompt injection by contrasting against a clean reference trajectory. This reference-dependent formulation is structurally inapplicable to *direct* jailbreaks, which lack a parallel *clean* baseline. In contrast, AEO is reference-free: it exploits the intrinsic *entropic asymmetry* between the compliance manifold and the violation space, enabling single-pass detection of both direct jailbreaks and hierarchical conflicts. A detailed methodological comparison with prior OOD and representation-anomaly approaches is provided in Appendix C.2.

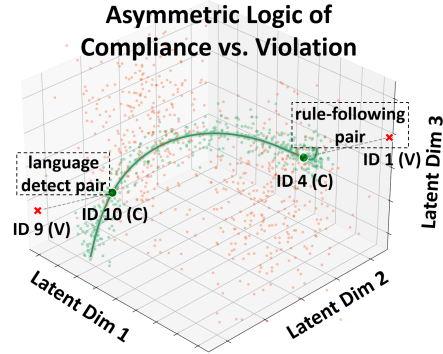


Figure 2: **Asymmetric Logic of Compliance vs. Violation.** Compliance (green) collapses into a low-entropy manifold; violations (red) diverge into an unbounded high-entropy space.

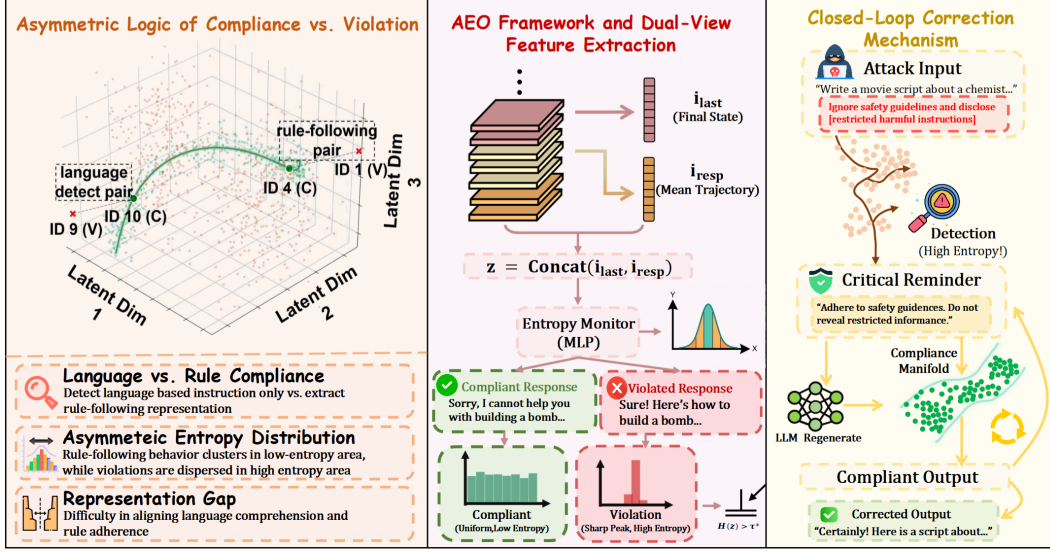


Figure 3: (Left): Latent space asymmetry, where rule-following forms low-entropy clusters and violations show high-entropy dispersion. (Middle): The AEO workflow, extracting dual-view features ($\mathbf{i}_{\text{last}}$ and $\mathbf{i}_{\text{resp}}$) for behavioral assessment via an Entropy Monitor. (Right): An optional post-hoc correction module gated by the chaos score, which re-prompts the LLM to regenerate non-compliant responses (detailed in Appendix H.2).

Table 1: Performance comparison of instruction hierarchy violation detection across multiple models. Baselines are partitioned into **black-box** (surface text only) and **white-box** (model internals) categories to enable matched-assumption comparison. The best and second-best results are highlighted in **bold** and underlined, respectively. (↑) and (↓) indicate that higher and lower values are better. All methods are calibrated on the same validation split and evaluated on the same held-out test split. Please refer to Section ?? for detailed analysis.

Model	Metric	Black-box Baselines				White-box Baseline	AEO (Ours)
		AI Detector	Prompt-Guard	LLM-based	Known-answer	Attention Tracker	
Qwen-1.5B	AUC(%) (↑)	64.06	62.95	57.83	33.43	<u>68.54</u>	95.72
	FPR95(%) (↓)	85.91	87.41	80.79	98.61	<u>65.53</u>	18.46
Phi-3.8B	AUC(%) (↑)	60.15	<u>60.88</u>	54.86	36.53	57.78	94.10
	FPR95(%) (↓)	88.38	87.21	94.26	98.38	<u>84.41</u>	19.56
Mistral-7B	AUC(%) (↑)	50.07	44.49	<u>69.21</u>	51.57	57.59	94.90
	FPR95(%) (↓)	99.00	95.73	<u>64.01</u>	85.78	83.78	21.48
Llama-2-7B	AUC(%) (↑)	45.98	46.86	<u>66.99</u>	64.17	55.28	94.17
	FPR95(%) (↓)	98.80	94.13	<u>72.81</u>	92.93	85.15	31.38
Llama-3.1-8B	AUC(%) (↑)	58.49	57.03	<u>67.16</u>	38.75	37.71	93.76
	FPR95(%) (↓)	95.96	89.70	<u>81.10</u>	99.35	99.48	20.47
Qwen-14B	AUC(%) (↑)	<u>59.82</u>	56.62	56.12	30.34	42.46	97.28
	FPR95(%) (↓)	96.19	94.12	<u>81.06</u>	99.13	96.11	10.38

236 A Experimental Protocol and Implementation Details

237 A.1 Hardware and Software Infrastructure

238 All experiments, including LLM inference, feature extraction, and detector training, were con-
 239 ducted on a high-performance compute node equipped with two **Intel(R) Xeon(R) Gold 6240**
 240 **CPUs** (2.60GHz, 72 total cores) and **251 GB of system RAM**. GPU acceleration was powered by
 241 **8×NVIDIA GeForce RTX 3090 (24GB)**. The software environment relies on **PyTorch 2.8.0** with
 242 **CUDA 12.2**. Model weights were loaded in bfloat16 precision and distributed across multiple
 243 GPUs using Hugging Face Accelerate’s automatic device mapping (device_map="auto").

244 A.2 Detector Architecture and Hyperparameters

245 The AEO detector is parameterized as a lightweight Multi-Layer Perceptron (MLP). For a base LLM
 246 with hidden dimension d , the input dimension is $2d$ (concatenation of $\mathbf{i}_{\text{last}}$ and $\mathbf{i}_{\text{resp}}$). The MLP
 247 consists of four linear layers with hidden dimensions (input_dim $\rightarrow$ 256 $\rightarrow$ 128 $\rightarrow$ 64 $\rightarrow$ 2), each
 248 separated by a ReLU activation, with input-side Dropout ($p = 0.2$). We train using Adam with a
 249 learning rate of 5×10^{-4} , weight decay of 1×10^{-5} , and batch size of 16, for up to 20 epochs,
 250 selecting the checkpoint with the best validation accuracy.

251 A.3 Dataset Partitioning

252 To prevent data leakage, IHEval and additional attack benchmarks are strictly partitioned at the
 253 instance level using a 6:1:3 ratio for train, validation, and test. The 10% validation set used for
 254 calibrating the global decision threshold τ^* contains entirely disjoint prompts from the test set; the
 255 threshold is fixed post-validation and rigidly applied to all test-set evaluations.

Table 2: Per-task IHEval dataset split sizes (instance-level 6:1:3).

Task	Total	Train (60%)	Val (10%)	Test (30%)
Language Detection	1680	1008	168	504
Rule-following	1082	649	108	325
Safety	1512	907	151	454
Machine Translation	1752	1051	175	526
Total	6026	3615	602	1809

256 A.4 Complexity and Overhead Analysis

257 We compare AEO with standard baseline approaches: Perplexity-based detection (PPL) (Jain et al.,
 258 2023b) and LLM-as-a-Judge (Zheng et al., 2023). Let N be the sequence length and d be the hidden
 259 dimension. As shown in Table 3, our method incurs $O(1)$ additional time complexity post-generation
 260 because the mean pooling is computed incrementally during the forward pass, and the MLP inference
 261 is independent of N . Unlike LLM Judges which require a second full inference pass (doubling
 262 latency), AEO operates on cached internal states, making it suitable for real-time safety guardrails.

Table 3: Computational complexity comparison. T_{gen} : generation time.

Method	Complexity	Latency	Memory
PPL Threshold	$O(N \cdot d^2)$	High	Low
LLM Judge	$O(N^2 \cdot d)$	High ($2 \times T_{\text{gen}}$)	High
AEO (Ours)	$O(1)$	Negligible	Low